

Dokumentacja Techniczna Systemu Zarządzania Kliniką

Architektura, API oraz Wymagania Funkcjonalne

Jakub Peska

29 maja 2026

Spis treści

1	Wprowadzenie	2
1.1	Cel dokumentu	2
1.2	Założenia architektoniczne	2
2	Wymagania Funkcjonalne	3
2.1	Wstęp	3
2.2	Przypadki Użycia i Zachowanie Systemu	3
2.2.1	Zarządzanie Wizytami (Appointments/Visits)	3
2.2.2	Zarządzanie Lekarzami	3
2.2.3	Zarządzanie Pacjentami	3
2.2.4	System Audytu (Blackbox) i Inicjalizacja	4
2.3	Podsumowanie modułu	4
3	Diagram Bazy Danych	5
3.1	Wstęp	5
3.2	Schemat ERD (Entity-Relationship Diagram)	5
3.3	Szczegóły struktury i indeksowanie	5
4	Specyfikacja API Backendu	7
4.1	Wstęp	7
4.2	Kluczowe Endpointy	7
4.2.1	Aktualizacja Wizyty	7
4.2.2	Usuwanie Lekarza	8
4.3	Podsumowanie modułu	8

Rozdział 1

Wprowadzenie

1.1 Cel dokumentu

Niniejszy dokument stanowi autorytatywną specyfikację techniczną systemu do zarządzania placówką medyczną. Obejmuje on zbiór wymagań funkcjonalnych, model relacyjny bazy danych oraz interfejs programistyczny aplikacji (API) dla backendu.

1.2 Założenia architektoniczne

Fundamentem architektury opisywanego systemu jest absolutna transparentność wszystkich procesów przetwarzania danych. System został zaprojektowany z myślą o zapewnieniu pełnej widoczności realizowanych operacji, co niweluje zjawisko "ukrytych błędów" lub nieautoryzowanych modyfikacji. Narzędziem realizującym to założenie jest wbudowany, kuloodporny `AuditService`, który działa analogicznie do czarnej skrzynki, utrwalając każdy stan encji przed i po jej modyfikacji.

Rozdział 2

Wymagania Funkcjonalne

2.1 Wstęp

Moduł ten opisuje wymagania funkcjonalne postawione przed aplikacją ze szczególnym uwzględnieniem zarządzania zasobami ludzkimi (pacjenci, lekarze) oraz wizytami, a także definiuje zasady bezpieczeństwa (autoryzacja na podstawie ról).

2.2 Przypadki Użycia i Zachowanie Systemu

2.2.1 Zarządzanie Wizytami (Appointments/Visits)

- **Modyfikacja Wizyty:** Uprawnieni użytkownicy (Rola 0 - Administrator, Rola 2 - Recepcja) mogą edytować istniejące wizyty za pomocą dedykowanych okien dialogowych (Angular Material).
- **Zarządzanie statusem:** Wizyty posiadają zdefiniowaną maszynę stanów: PLANNED, COMPLETED, CANCELLED. Na warstwie prezentacji stany te są wyróżniane za pomocą dedykowanej palety barw.
- **Śledzenie zmian:** Każda edycja wizyty jest przechwytywana, a stan poprzedni i nowy jest utrwalany w logach audytu.

2.2.2 Zarządzanie Lekarzami

- **Podgląd harmonogramu:** Użytkownik posiada dostęp do kalendarza pracy wybranego lekarza.
- **Usuwanie konta lekarza:** Operacja usunięcia lekarza z systemu wymusza zastosowanie kaskadowego usuwania przypisanych mu wizyt. Zapobiega to naruszeniu więzów spójności (Foreign Key Constraint). Operacja usunięcia całego węzła jest raportowana w audycie. Wyłącznie administratorzy (Rola 0) posiadają uprawnienia do tej akcji.

2.2.3 Zarządzanie Pacjentami

- **Zarządzanie danymi:** Edycja profilu pacjenta (imię, nazwisko, PESEL, numer telefonu).

- **Zabezpieczenie przed duplikacją:** PESEL posiada unikalny klucz (UNIQUE), wymuszający weryfikację na poziomie silnika bazy danych.

2.2.4 System Audytu (Blackbox) i Inicjalizacja

- **Logowanie Akcji:** Moduł bezwzględnie rejestrujący żądania modyfikujące stan (PUT, DELETE). Zapisuje kto, kiedy, dlaczego oraz co zmienił.
- **Masowe zasilanie danymi (Seeder):** System jest zoptymalizowany pod obciążenia. Moduł `createdb.mjs` integruje `FakerPL` w celu transakcyjnego zasilenia bazy minimum 10 000 pacjentów oraz 50 000 wizyt, generując potężny wolumen danych w kilka sekund dzięki instrukcjom `BEGIN` i `COMMIT`.

2.3 Podsumowanie modułu

System realizuje pełen cykl życia obiektów medycznych, kładąc szczególny nacisk na asynchroniczne odświeżanie interfejsu za pomocą mechanizmów detekcji zmian (`ChangeDetection`) w frameworku Angular oraz na spójność danych.

Rozdział 3

Diagram Bazy Danych

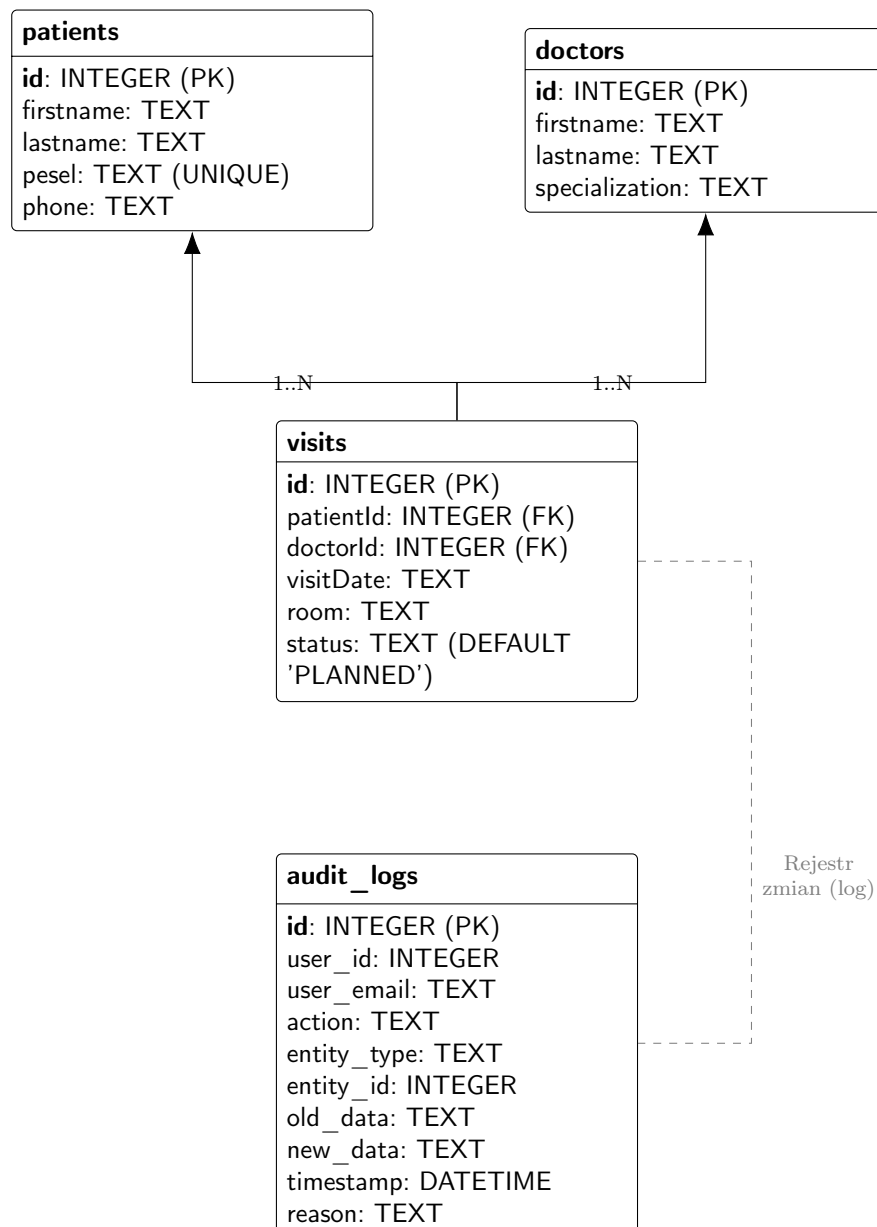
3.1 Wstęp

Silnikiem bazodanowym systemu jest relacyjna baza `SQLite`. Skrypt inicjalizacyjny zapewnia budowę struktur opartych o rygorystyczne więzy integralności (Foreign Keys = ON).

3.2 Schemat ERD (Entity-Relationship Diagram)

3.3 Szczegóły struktury i indeksowanie

Tabela `audit_logs` posiada indeks `idx_audit_entity` obejmujący kolumny `entity_type` oraz `entity_id`. Ograniczenie to gwarantuje subsekundowe czasy odpowiedzi podczas generowania historii zmian dla danego zasobu (pacjenta lub wizyty), nawet przy dziesiątkach tysięcy wpisów. Zastosowano konwencję nazewnictwa kluczy obcych camelCase (np. `patientId`) dla ujednolicenia z interfejsem warstwy backendu (Node.js/Express).



Rysunek 3.1: Struktura relacyjna bazy danych SQLite

Rozdział 4

Specyfikacja API Backendu

4.1 Wstęp

Warstwa serwerowa została zrealizowana w środowisku Node.js z wykorzystaniem frameworka Express. Komunikacja odbywa się w standardzie REST, a ładunki użyteczne wymieniane są w formacie JSON. Wszystkie ścieżki podlegają uwierzytelnieniu za pośrednictwem odpowiednich middleware (funkcja `requireAuth`).

4.2 Kluczowe Endpointy

4.2.1 Aktualizacja Wizyty

Służy do edycji parametrów zaplanowanej lub zakończonej wizyty. Endpoint wbudowuje mechanizm automatycznego zapisu do audytu.

Metoda HTTP	PUT
Ścieżka	/api/visits/:id
Wymagane role	0 (Admin), 2 (Recepcja)
Parametry ścieżki	id (Number) - Unikalny identyfikator wizyty.
Request Body 200 OK - {"message": "Wizyta zaktualizowana pomyślnie"}	
Możliwe błędy	404 Not Found (Gdy wizyta nie istnieje), 500 Internal Server Error (Błąd constraintów SQL)

4.2.2 Usuwanie Lekarza

Wykonuje trwałe usunięcie zasobu lekarza z bazy danych wraz z **kaskadowym usunięciem** powiązanych wizyt.

Metoda HTTP	DELETE
Ścieżka	/api/doctors/:id
Wymagane role	0 (Admin)
Parametry ścieżki	id (Number) - Unikalny identyfikator lekarza.
Logika Biznesowa	Silnik najpierw egzekwuje <code>DELETE FROM visits WHERE doctorId = ?</code> , zwalniając blokadę Foreign Key, po czym usuwa rekord z tabeli <code>doctors</code> . Następuje zapis encji usuniętej do audytu z odpowiednią flagą.
Odpowiedź (Sukces)	200 OK - {"message": "Lekarz i jego wizyty zostali trwale usunięci z systemu."}

4.3 Podsumowanie modułu

Architektura interfejsu programistycznego rygorystycznie chroni integralność relacyjną danych. Middleware autoryzacyjne działają jak strażnik dostępu, a serwis logujący gwarantuje odporność na uszkodzenia operacyjne w sytuacji, gdy rekord źródłowy zostaje usunięty.

Bibliografia

- [1] Zespół Google, *Angular Documentation*, 2026.
- [2] D. Richard Hipp, *SQLite Database Architecture*, 2026.
- [3] Faker.js Community, *FakerPL - Generowanie zbiorów testowych*, 2026.

Skorowidz

AuditService, [2](#)

Endpoint, [7](#)

Faker, [4](#)

Foreign Key, [3](#)

Kaskadowe Usuwanie, [8](#)

Material Design, [3](#)

REST API, [7](#)

SQLite, [5](#)